

Angular Avançado · Módulos 1 e 7

Sessão 13

Sheriff e NgRx Signal Store

João Gonçalves · joaogoncalves.net · Making the complicated simple

Recap · Três dependency rules

importa →	feature	ui	data	util
feature	—	✓	✓	✓
ui	✗	—	✓	✓
data	✗	✗	—	✓
util	✗	✗	✗	✗ ?

Domains

O próprio + `shared`. Nunca outro domain diretamente.

API pública

Sem `index.ts`: tudo é público **exceto** a subpasta `internal/`.

A linha importa a coluna. `ui → data` ✓ (o `FlightCard` recebe um `Flight`) · `ui → feature` ✗.

Recap · O Skyboard na architecture matrix

Organizado assim desde a Sessão 1

	ticketing	shared
feature	`FlightSearch` (+ `FlightStore`) · `FlightEdit` · `BoardingList` · `BoardingLocked`	
ui	`FlightCard` · `PassengerRow`	`SimpleDelayStepper` · `ValidationPane` `loggingInterceptor ?`
data	`Flight` · `FlightClient` · `flightSchema` · `validateRoundTrip`	
util		`AuthStore` · `authGuard` · `authInterceptor` · `ConfigService` · `exitGuard`

A `shell` (navbar, home, about) fica fora: é a raiz e pode usar tudo.

Agenda de hoje

01	**Sheriff**	As dependency rules verificadas no Skyboard	≈ 40 min
02	**Local ou partilhado?**	Feature slicing · stores · modulith	≈ 12 min
03	**NgRx Signal Store**	A FlightStore com withState · withComputed · withMethods	≈ 50 min
04	**Resumo**	E o que fica para a Sessão 14	≈ 5 min



Bloco C

As regras, verificadas

Dependency rules no papel esquecem-se — uma ferramenta lê os imports

O que já é verificado — e o que não é

Ferramentas que verificam regras por nós

TypeScript ✓

verifica os tipos

string ≠ number

ESLint ✓

verifica o estilo

variável não usada

??? ✗

quem pode importar quem?

ninguém verifica

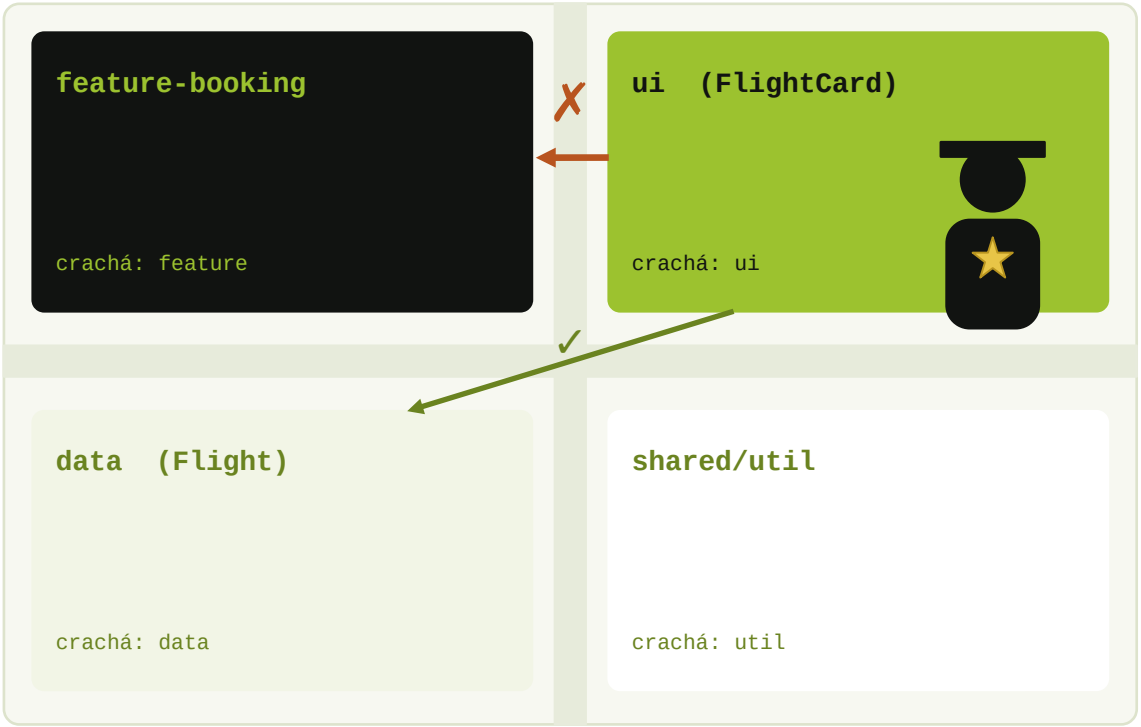
```
// ticketing/ui/flight-card/flight-card.ts — um componente de apresentação
import { FlightStore } from '../../feature-booking/flight-search/flight-store';
```

Compila ✓ Funciona ✓ Code review… ninguém deu por isso

Regras no papel **esquecem-se**: alguém novo, pressa, "só desta vez" — e em seis meses voltámos ao esparguete.

O Sheriff: o xerife da cidade

Bairros, crachás e uma lei — verificados automaticamente



Bairros

as nossas pastas

Crachás

as tags de cada pasta

A lei

as dependency rules

"Quem tem o crachá **ui** não entra em bairros **feature**."

O que é — e o que não é

É…

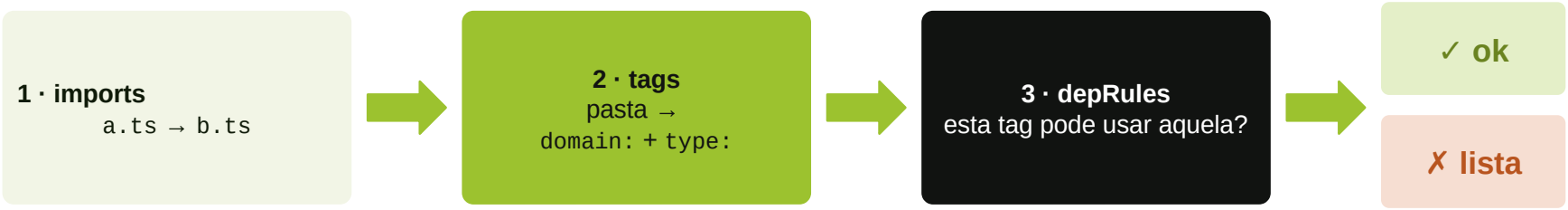
- Uma ferramenta **open source** que lê os imports
- Compara cada import com **as regras que escrevemos**
- Corre como um **teste**: no CI, o build falha

Não é…

- Um framework: **não muda a app** (só `devDependency`, fora do bundle)
- Uma reorganização: descreve as **pastas que já temos**
- Quem decide: **nós** escrevemos as regras

O TypeScript verifica os **tipos**, o ESLint o **estilo** — o Sheriff verifica as **fronteiras**.

Como funciona o Sheriff




```
src/app/domains/ticketing/ui/flight-card/flight-card.ts
  tags: domain:ticketing, type:ui
  importa → feature-booking/flight-search/flight-store.ts
  tags: domain:ticketing, type:feature
  type:ui → type:feature ?   ✗  Dependency Rule Violation
```

Na linha de comandos (`npx sheriff verify`, dá para CI) ou no ESLint (erros no editor).

Sheriff: as regras verificadas

sheriff.config.ts

```ts
export const config: SheriffConfig = {
  enableBarrelLess: true,
  modules: {                          // pastas → tags
    'src/app/domains/<domain>': {
      'feature-<name>': ['domain:<domain>', 'type:feature'],
      ui: ['domain:<domain>', 'type:ui'],  …
    },
  },
  depRules: {                         // tag → o que pode usar
    root: '*',
    'domain:*': [sameTag, 'domain:shared'],
    'type:feature': ['type:ui', 'type:data', 'type:util'],  …
  },
};
```

Corre…

- `npx sheriff verify` (linha de comandos, CI)
- `npx sheriff list` (que tags tem cada pasta)
- ou no **ESLint** (erros no editor)



Snippet A1 · da teoria ao código

Agora, vamos escrever o código

- `npm i -D @softarc/sheriff-core`
- Criar o `sheriff.config.ts` na raiz: as pastas (tags) e as regras
- Experimentar: `npx sheriff list` — 8 pastas, cada uma com o seu crachá

package.json · sheriff.config.ts (raiz do projeto)

A1 · Instalar e descrever a matriz

```
npm i -D @softarc/sheriff-core
```

sheriff.config.ts (raiz do projeto)

```ts
import { sameTag, SheriffConfig } from '@softarc/sheriff-core';
export const config: SheriffConfig = {
  entryFile: 'src/main.ts',
  enableBarrelLess: true,
  modules: {
    'src/app/domains/<domain>': {
      'feature-<name>': ['domain:<domain>', 'type:feature'],
      'ui-<name>': ['domain:<domain>', 'type:ui'],
      ui: ['domain:<domain>', 'type:ui'],
      data: ['domain:<domain>', 'type:data'],
      'util-<name>': ['domain:<domain>', 'type:util'],
    },
  },
  depRules: {
    root: '*',
    'domain:*': [sameTag, 'domain:shared'],
    'type:feature': ['type:ui', 'type:data', 'type:util'],
    'type:ui': ['type:data', 'type:util'],
    'type:data': ['type:util'],
    'type:util': [],
  },
};
```

Experimentar

- `npx sheriff list`
- 8 pastas, cada uma com `domain:` e `type:`



Snippet A2 · da teoria ao código

Agora, vamos escrever o código

- `npx sheriff verify` — o que encontra no Skyboard?
- Uma violação que já lá estava: discutir com a sala
- Decidir: ajustar a regra `'type:util'` → `No issues found`

terminal · sheriff.config.ts

A2 · O que o Sheriff encontra

```
$ npx sheriff verify

Issues found:
  Total Dependency Rule Violations: 1

|-- src/app/domains/shared/util-auth/auth-interceptor.ts
|   |-- Dependency Rule Violations
|   |   |-- from tag type:util to tags domain:shared, type:util
```

Já lá estava: util-auth → util-common

- O `authInterceptor` lê o `apiUrl` do `ConfigService`
- O que fazemos? Mover · passar de outra forma · **ajustar a regra**

```
// a equipa decide:
'type:util': ['type:util'],
```



Snippet A3 · da teoria ao código

Agora, vamos escrever o código

- No `FlightCard`, importar a `FlightStore` (só o import)
- `npx sheriff verify` → `type:ui` → `type:feature` ✗
- Desfazer o import

ticketing/ui/flight-card/flight-card.ts

A3 · Provocar uma violação

```
// ticketing/ui/flight-card/flight-card.ts — só para ver o Sheriff a reagir
import { FlightStore } from '../../feature-booking/flight-search/flight-store';
```

```
$ npx sheriff verify
|-- src/app/domains/ticketing/ui/flight-card/flight-card.ts
|   |   |-- from tag type:ui to tags domain:ticketing, type:feature
```

Porque é grave?

O `FlightCard` deixava de ser reutilizável: ficava preso a **um** caso de uso.

E também…

Importar algo de `data/internal/` → **Encapsulation Violation**.



Snippet A4 · da teoria ao código

Agora, vamos escrever o código

- Mover o `logging-interceptor.ts` para `shared/util-common/`
- Corrigir o import no `app.config.ts`
- Script `lint:arch` no `package.json` → `npm run lint:arch`

shared/util-common/logging-interceptor.ts · app.config.ts · package.json

A4 · Arrumar e automatizar

```
git mv src/app/domains/shared/ui-common/logging-interceptor.ts \
       src/app/domains/shared/util-common/
```

```
// app.config.ts
import { loggingInterceptor } from
  './domains/shared/util-common/logging-interceptor';
```

```
// package.json → scripts
"lint:arch": "sheriff verify"

$ npm run lint:arch   → No issues found.
```

O Sheriff verifica as **regras**; o **nome** da pasta continua a ser uma decisão nossa.

Um interceptor em `ui-common` não viola nenhuma regra (ui pode usar util) — mas engana quem procura.

Na vida real

Path mappings · ESLint · Detective

```
// tsconfig.json → compilerOptions
"paths": { "@skyboard/*": ["./src/app/domains/*"] }

// antes
import { FlightClient } from '../../data/flight-client';
// depois: domínio + módulo à vista
import { FlightClient } from '@skyboard/ticketing/data/flight-client';
```

ESLint

`@softarc/eslint-plugin-sheriff`: as violações aparecem **no editor**. Exige ESLint (`ng add angular-eslint`).

Detective

`npx detective`: o grafo de dependências no browser. Bom para conhecer um projeto novo.

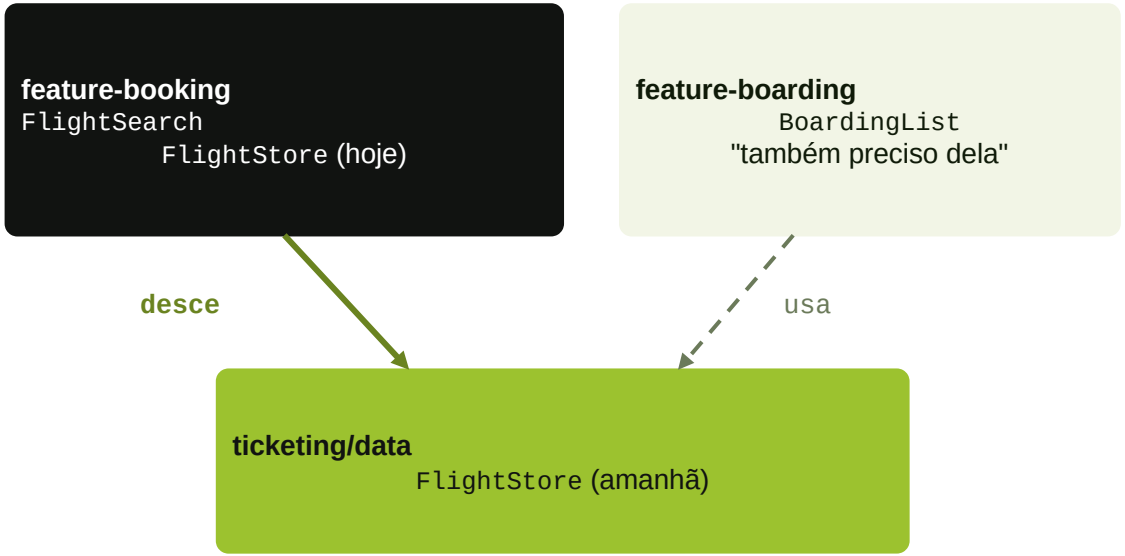


Bloco D

Local ou partilhado?

Feature slicing · stores · modulith

Feature slicing: tão local quanto possível



Regra prática

- Começa na feature
- Uma 2.ª feature precisa → desce para `ui` ou `data`
- Exige uma equipa atenta ao momento de refatorar

Tão local quanto possível, tão global quanto necessário.

Stores e arquitetura



Um só sentido: intenção → processar → estado → vista

Onde vive?

- **feature**: o caso de uso (a nossa `FlightStore`)
- **data**: partilhada no domínio
- **util**: genérica (a `AuthStore`)
- Começa local; desce quando outra feature precisar

Stores a falar com stores?

- Acoplamento e ciclos
- Só ler de uma camada **abaixo**
- Ou uma **facade** que orquestra
- Ou eventos → NgRx (Sessão 13)

Modulith ou micro frontends?

Modulith

- Uma codebase, módulos, **um** deploy
- Fronteiras protegidas por linting
- Fácil partilhar e refatorar
- A melhor escolha para 1–poucas equipas

Micro frontends

- Várias codebases, deploy **independente**
- Autonomia, até stacks diferentes
- Partilhar é mais difícil · mais infraestrutura
- Muitas equipas, sistemas grandes, anos

O Skyboard é um **modulith**.



Bloco E

NgRx Signal Store

A nossa FlightStore — agora com uma biblioteca

A FlightStore de hoje: feita à mão

feature-booking/flight-search/flight-store.ts

```
@Service()
export class FlightStore {
  private readonly _from = signal('Lisbon');
  readonly from = this._from.asReadonly();
  // … o mesmo para to e basket

  private readonly flightsResource =
    this.flightClient.findResource(this.from, this.to);
  readonly flights = this.flightsResource.value;

  updateFilter(from: string, to: string) {
    this._from.set(from); this._to.set(to);
  }
}
```

O padrão

- Estado: `signal` privado + `asReadonly()`
- Dados do backend: um `resource`
- Métodos que **escrevem** o estado
- Componente só **lê** e chama métodos

Funciona — mas em cada store escrevemos **o mesmo código** à mão.

NgRx Signal Store: a mesma ideia, por peças

signalStore(…) — cada peça vê o que as anteriores criaram

withState	withProps	withComputed	withMethods
o estado (signals só de leitura)	o que não é estado (resource, serviços)	o estado derivado (computed)	as operações (patchState)

```
export const FlightStore = signalStore(
  { providedIn: 'root' },
  withState(initialState),
  withProps(…), withComputed(…), withMethods(…),
);
```

Para quem usa

`inject(FlightStore)` — **igual** a um serviço.

Pacote @ngrx/signals — já está no Skyboard desde o início.

withState e patchState

O estado lê-se como signals; escreve-se só com patchState

```
withState({ from: 'Lisbon', to: 'Porto', basket: {} })

// fora: store.from()  store.to()  store.basket()

withMethods((store) => ({
  updateFilter(from: string, to: string) {
    patchState(store, { from, to });
  },
  updateBasket(id: number, selected: boolean) {
    patchState(store, (state) => ({
      basket: { ...state.basket, [id]: selected },
    }));
  },
}))
```

Ler

- Cada propriedade → um signal
- **Só de leitura** fora da store

Escrever

- `patchState`: um objeto **ou** uma função
- Imutável: um objeto novo, nunca mutar



Snippet N1 · da teoria ao código

Agora, vamos escrever o código

- **Substituir** a `FlightStore`: `withState` · `withProps` (o resource) · `withMethods`
- O `FlightSearch` **não muda**: `inject(FlightStore)`, os mesmos nomes
- Experimentar: pesquisar, mudar o cesto — tudo igual

feature-booking/flight-search/flight-store.ts

N1 · A FlightStore com a NgRx

Os mesmos nomes por fora — por dentro, peças

```
export const FlightStore = signalStore(
  { providedIn: 'root' },
  withState(initialState),
  withProps((store) => ({
    _flightsResource: inject(FlightClient).findResource(store.from, store.to),
  })),
  withProps((store) => ({
    flights: store._flightsResource.value, error: …, isLoading: …, status: …,
  })),
  withMethods((store) => ({
    updateFilter(…) { patchState(store, { from, to }); },
    updateBasket(…) { patchState(store, (state) => ({ … })); },
    reload() { store._flightsResource.reload(); },
  })),
);
```

_ no nome = **privado**: só as peças da store o veem. O componente vê `flights`, não o resource.

withComputed: estado derivado

O que se calcula a partir do estado — dentro da store

```
withComputed((store) => ({
  route: computed(() => `${store.from()} → ${store.to()}`),
  selectedCount: computed(() =>
    Object.values(store.basket()).filter(Boolean).length
  ),
})),
```

Porquê aqui?

- Qualquer componente que use a store recebe o mesmo valor
- O componente fica **só a mostrar**

```
// flight-search.ts — antes
flightRoute = computed(() => `${this.store.from()} → …`);
// depois
flightRoute = this.store.route;
```

Mesma regra das Sessões 3–4: o que se deriva é computed, não é estado.



Snippet N2 · da teoria ao código

Agora, vamos escrever o código

- Na store: **acrescentar** `withComputed` com `route` e `selectedCount`
- No `FlightSearch`: `flightRoute = this.store.route` (sai o `computed`)
- No template: mostrar quantos voos estão no cesto

flight-store.ts · flight-search.ts · flight-search.html

N2 · Estado derivado na store

O componente fica mais simples

```
// flight-search.ts
protected readonly flightRoute = this.store.route;
protected readonly selectedCount = this.store.selectedCount;

<!-- flight-search.html -->
<h2>{{ flightRoute() }} · {{ selectedCount() }} no cesto</h2>
```

Marcar um voo no cesto

`updateBasket` → `patchState` → `basket` muda → `selectedCount` recalcula → o template atualiza.

Um só sentido

Intenção (método) → estado → derivado → vista. O componente nunca escreve no estado.

À mão ou com a NgRx?

As mesmas ideias — menos código repetido

	Classe feita à mão	NgRx Signal Store
Estado	signal privado + asReadonly()	withState — só de leitura por omissão
Escrever	set / update em cada signal	patchState, imutável
Derivado	computed no componente ou na classe	withComputed
Privado	private	prefixo _
Usar	inject(FlightStore)	inject(FlightStore) — igual

Para uma store pequena, a classe à mão chega. A NgRx compensa quando há **muitas stores** e a equipa quer **um só padrão**.

A store e as dependency rules

A NgRx não muda as regras de ontem

```
$ npm run lint:arch
No issues found. Well done!
```

Onde vive

- Hoje: em `feature-booking` (só o `FlightSearch` a usa)
- Se o boarding precisar: desce para `ticketing/data`
- Importa só para baixo: `data` (o `FlightClient`)

Na Sessão 14

- `rxMethod`: RxJS dentro da store (debounce, switchMap)
- `toSignal` · `toObservable`
- `withHooks`: carregar ao iniciar
- Eventos entre stores

Resumo da sessão

Sheriff

- `sheriff.config.ts`: tags + `depRules`
- `npm run lint:arch` (CI)
- Exceções = decisões escritas

Local ou partilhado

- Começa na feature
- Desce quando outra precisar
- Tão local quanto possível

NgRx Signal Store

- `withState` · `withProps`
- `withComputed` · `withMethods`
- `patchState`: imutável

As regras dizem **onde** vive cada peça — a Signal Store dá **uma forma** a cada store.



Na Sessão 14 · a última

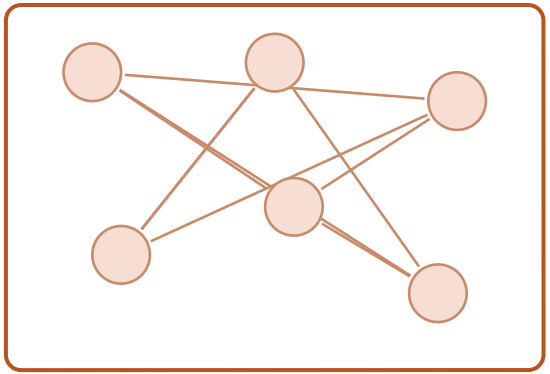
RxJS na store e performance

- `rxMethod` · `toSignal` · `toObservable` · `withHooks`
- `@defer` · zoneless · uma visão de SSR
- Fecho do curso

Extra · Do monólito ao modulith

Fronteiras e deploys: três formas de organizar uma app

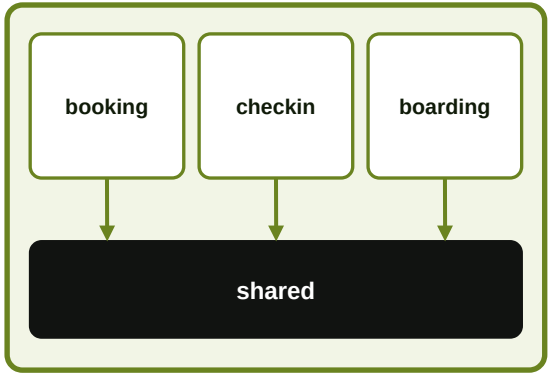
Big ball of mud



Fronteiras: **nenhumas**

Deploy: **um**

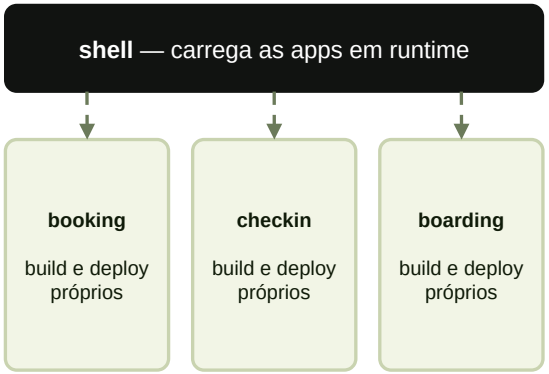
Modulith



Fronteiras: **claras e verificadas**

Deploy: **um**

Micro frontends



Fronteiras: **físicas**

Deploys: **vários**

O problema nunca foi ter **um só deploy** — foi **não ter fronteiras**.

O Skyboard é um modulith. Um módulo bem isolado extrai-se mais tarde para micro frontend, sem reescrever.

Extra · Monorepo e micro frontends

Onde guardas o código ≠ como o entregas

Monorepo: onde está o código

- Um repo, várias apps e libraries
- Partilhar e refatorar fácil · uma versão de cada dependência
- **Nx**, Turborepo: build só do que mudou, cache
- Fronteiras: `enforce-module-boundaries` (como o Sheriff)

Micro frontends: como se entrega

- Várias apps, deploys independentes, uma shell
- Autonomia total por equipa
- Custo: infraestrutura, versões, partilha difícil
- Angular: **Module Federation** · **Native Federation**

	Um deploy	Vários deploys
Um repo	Modulith num monorepo — ou só uma app (o Skyboard)	Micro frontends num monorepo (comum com Nx)
Vários repos	Raro	Micro frontends "clássicos" (polyrepo)

Monorepo é sobre **onde guardas** o código; micro frontends, sobre **como o entregas**.